

UNIT 1: COLLECTION FRAMEWORK

1.1 Collection Interface

The **Collection Framework** in Java is a unified architecture used to store, manage, and manipulate groups of objects.

It provides:

- Interfaces
- Classes
- Algorithms
- Utility methods

The main interfaces in the Java Collection Framework are:

1. **Collection**
2. **List**
3. **Set**
4. **Queue**
5. **Map**

Collection Interface

The **Collection** interface is the root interface of the collection hierarchy. It represents a group of objects known as elements.

It provides common methods for adding, removing, searching, and checking elements.

Common Methods of Collection Interface

Method	Description
add()	Adds an element
addAll()	Adds all elements from another collection
remove()	Removes an element
removeAll()	Removes all matching elements
contains()	Checks whether an element exists
size()	Returns the number of elements
isEmpty()	Checks whether the collection is empty
clear()	Removes all elements
iterator()	Returns an iterator

Example

```
import java.util.*;

class CollectionExample {
    public static void main(String[] args) {
        Collection<String> names = new ArrayList<>();

        names.add("Rahul");
        names.add("Amit");
        names.add("Priya");

        System.out.println(names);
        System.out.println(names.size());
        System.out.println(names.contains("Amit"));
    }
}
```

Advantages of Collection Framework

- Reduces programming effort.
 - Provides ready-to-use data structures.
 - Provides efficient algorithms.
 - Supports dynamic storage.
 - Improves code reusability.
 - Provides standard methods for data manipulation.
-

1.2 ArrayList and Vector

ArrayList

ArrayList is a resizable array implementation of the **List** interface.

Unlike a normal array, the size of an ArrayList can grow or shrink dynamically.

It allows:

- Duplicate elements.
- null values.

- Random access using an index.
- Maintaining insertion order.

Example

```
import java.util.*;

class ArrayListExample {
    public static void main(String[] args) {
        ArrayList<String> names = new ArrayList<>();

        names.add("Amit");
        names.add("Rahul");
        names.add("Amit");

        System.out.println(names);
    }
}
```

Output:

[Amit, Rahul, Amit]

Common ArrayList Methods

Method	Description
add()	Adds an element
get()	Returns an element at an index
set()	Updates an element
remove()	Removes an element
size()	Returns the number of elements
contains()	Checks whether an element exists
clear()	Removes all elements

Advantages of ArrayList

- Dynamic size.
- Fast random access.

- Allows duplicates.
- Easy to use.
- Efficient for retrieving elements.

Disadvantages

- Insertion and deletion in the middle can be slower.
- Not synchronized by default.

Vector

Vector is another resizable array implementation of the **List** interface.

It is similar to `ArrayList` but its methods are **synchronized**, making it thread-safe by default.

Example

```
import java.util.*;

class VectorExample {
    public static void main(String[] args) {
        Vector<Integer> numbers = new Vector<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        System.out.println(numbers);
    }
}
```

ArrayList vs Vector

Feature	ArrayList	Vector
Synchronization	Not synchronized	Synchronized
Performance	Generally faster	Generally slower
Thread Safety	Not thread-safe by default	Thread-safe by default
Introduced	Java 1.2	Java 1.0

Feature	ArrayList	Vector
Growth	Grows dynamically	Grows dynamically
Use	General-purpose applications Legacy/thread-safe scenarios	

1.3 Generics and Iterator

Generics

Generics allow classes, interfaces, and methods to work with different data types while providing compile-time type safety.

Generics are represented using angle brackets < >.

Example Without Generics

```
ArrayList list = new ArrayList();
```

```
list.add("Hello");
```

```
list.add(10);
```

Different types of data can be inserted, which may cause problems during type casting.

Example With Generics

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Amit");
```

```
names.add("Rahul");
```

Here, the ArrayList can store only **String** values.

Advantages of Generics

- Provides type safety.
- Reduces type casting.
- Detects errors at compile time.
- Improves code readability.
- Allows reusable code.

Generic Class Example

```
class Box<T> {
    T value;
```

```
void set(T value) {  
    this.value = value;  
}  
  
T get() {  
    return value;  
}  
}
```

Here, T is a type parameter.

Iterator

An **Iterator** is an interface used to traverse elements of a collection one by one.

It is available in the java.util package.

Common Iterator Methods

Method	Description
--------	-------------

hasNext()	Checks if another element exists
-----------	----------------------------------

next()	Returns the next element
--------	--------------------------

remove()	Removes the current element
----------	-----------------------------

Example

```
import java.util.*;  
  
class IteratorExample {  
    public static void main(String[] args) {  
        ArrayList<String> names = new ArrayList<>();  
  
        names.add("Amit");  
        names.add("Rahul");  
        names.add("Priya");  
  
        Iterator<String> itr = names.iterator();
```

```

        while (itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}

```

Advantages of Iterator

- Provides a standard way to traverse collections.
- Works with many collection classes.
- Allows safe removal of elements during traversal.
- Simple and easy to use.

1.4 HashSet, TreeSet and Comparable

HashSet

HashSet is a collection class that implements the **Set** interface.

It stores **unique elements** and does not allow duplicate values.

HashSet generally does not guarantee the order of elements.

Example

```

import java.util.*;

class HashSetExample {
    public static void main(String[] args) {
        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(30);
        numbers.add(10);
        numbers.add(20);
        numbers.add(10);

        System.out.println(numbers);
    }
}

```

```
}
```

The duplicate value **10** is stored only once.

Characteristics

- Does not allow duplicates.
 - Does not guarantee insertion order.
 - Allows one null element.
 - Provides efficient basic operations on average.
 - Uses hashing internally.
-

TreeSet

TreeSet is a collection class that implements the **NavigableSet** interface.

It stores unique elements in **sorted order**.

Example

```
import java.util.*;
```

```
class TreeSetExample {  
    public static void main(String[] args) {  
        TreeSet<Integer> numbers = new TreeSet<>();  
  
        numbers.add(30);  
        numbers.add(10);  
        numbers.add(20);  
  
        System.out.println(numbers);  
    }  
}
```

Output:

```
[10, 20, 30]
```

Characteristics

- Does not allow duplicates.
- Stores elements in sorted order.

- Generally does not allow null elements.
 - Uses a tree-based data structure internally.
-

Comparable

Comparable is an interface used to define the **natural ordering** of objects.

It belongs to the java.lang package.

A class implements Comparable and overrides the:

compareTo()

method.

Example

```
class Student implements Comparable<Student> {  
    int marks;  
  
    Student(int marks) {  
        this.marks = marks;  
    }  
  
    public int compareTo(Student s) {  
        return this.marks - s.marks;  
    }  
}
```

Here, students are compared based on their marks.

Return Value of compareTo()

- Negative value → Current object comes before the other object.
- Zero → Both objects are considered equal in ordering.
- Positive value → Current object comes after the other object.

Comparable vs TreeSet

TreeSet can use the natural ordering defined by Comparable to maintain elements in sorted order.

1.5 HashMap, Hashtable and TreeMap

HashMap

HashMap is a collection class that stores data in the form of **key-value pairs**.

Each key must be unique.

Example

```
import java.util.*;

class HashMapExample {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();

        students.put(101, "Amit");
        students.put(102, "Rahul");
        students.put(103, "Priya");

        System.out.println(students);
    }
}
```

Here:

- 101, 102, and 103 are keys.
- "Amit", "Rahul", and "Priya" are values.

Common HashMap Methods

Method	Description
put()	Adds or updates a key-value pair
get()	Returns the value associated with a key
remove()	Removes a key-value pair
containsKey()	Checks whether a key exists
containsValue()	Checks whether a value exists
size()	Returns the number of entries
clear()	Removes all entries

Characteristics

- Stores key-value pairs.

- Keys are unique.
 - Allows one null key.
 - Allows multiple null values.
 - Does not guarantee insertion order.
 - Not synchronized by default.
-

Hashtable

Hashtable is a legacy collection class that stores key-value pairs.

It is similar to HashMap but has some important differences.

Example

```
import java.util.*;

class HashtableExample {
    public static void main(String[] args) {
        Hashtable<Integer, String> students = new Hashtable<>();

        students.put(101, "Amit");
        students.put(102, "Rahul");

        System.out.println(students);
    }
}
```

Characteristics

- Stores key-value pairs.
 - Keys must be unique.
 - Does not allow null keys.
 - Does not allow null values.
 - Methods are synchronized.
 - It is a legacy class.
-

TreeMap

TreeMap is a map implementation that stores key-value pairs in **sorted order of keys**.

It is based on a tree structure.

Example

```
import java.util.*;

class TreeMapExample {
    public static void main(String[] args) {
        TreeMap<Integer, String> students = new TreeMap<>();

        students.put(103, "Priya");
        students.put(101, "Amit");
        students.put(102, "Rahul");

        System.out.println(students);
    }
}
```

Output:

```
{101=Amit, 102=Rahul, 103=Priya}
```

The entries are sorted according to their keys.

Characteristics

- Stores key-value pairs.
- Keys are unique.
- Keys are maintained in sorted order.
- Does not allow null keys in natural ordering.
- Provides navigation methods such as firstKey() and lastKey().

Difference Between HashMap, Hashtable and TreeMap

Feature	HashMap	Hashtable	TreeMap
Data	Key-Value	Key-Value	Key-Value

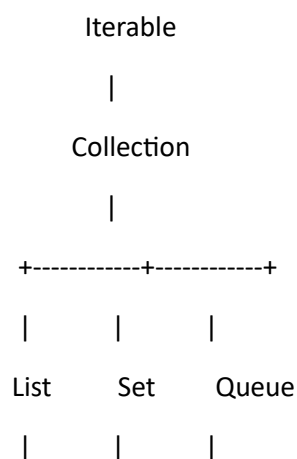
Feature	HashMap	Hashtable	TreeMap
Ordering	No guaranteed order	No guaranteed order	Sorted by keys
Synchronization	Not synchronized	Synchronized	Not synchronized
Null Key	Allows one	Not allowed	Not allowed with natural ordering
Null Values	Allowed	Not allowed	Generally allowed
Performance	Generally fast	Generally slower due to synchronization	Generally slower than HashMap
Type	Modern	Legacy	Sorted Map

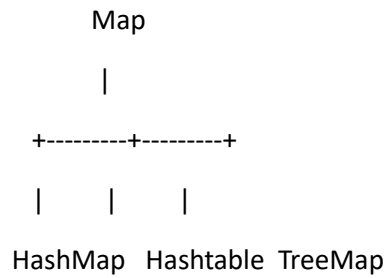
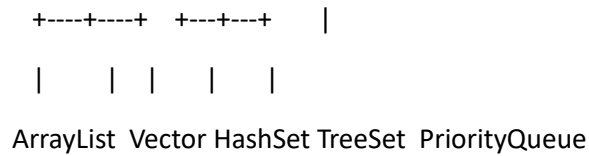
Difference Between HashSet and TreeSet

Feature	HashSet	TreeSet
Duplicate Elements	Not allowed	Not allowed
Ordering	No guaranteed order	Sorted order
Data Structure	Hash table-based	Tree-based
Performance	Generally faster	Generally slower
Null	Allows one null	Generally does not allow null
Use	Fast unique-element storage	Sorted unique-element storage

Collection Framework Hierarchy

A simplified view of the Java Collection Framework is:





Note: Map is part of the Java Collections Framework, but it does **not** extend the Collection interface.

Summary

The **Java Collection Framework** provides a standard way to store and manipulate groups of objects. The **Collection interface** provides common operations for collections. **ArrayList** and **Vector** are dynamic list implementations, while **Generics** provide type safety and **Iterator** allows collection traversal. **HashSet** stores unique elements without guaranteeing order, whereas **TreeSet** stores unique elements in sorted order. The **Comparable** interface defines natural ordering for objects. **HashMap** and **Hashtable** store key-value pairs, while **TreeMap** stores key-value pairs sorted by their keys. These classes and interfaces make Java programming easier, more flexible, and more efficient.